Advanced Database Systems Portfolio

Individual practical assessment summaryIndividual practical assessment summary

Student Osimen Odigie	Course MIT 8103 Advanced Database Systems
Student ID 301826111	Session 2026/2027, First Semester

Case study and scope

I used a Brazilian online marketplace based on the public Olist e-commerce dataset. Customers place orders for products offered by sellers, make one or more payments, receive deliveries and leave reviews. The database supports order processing, stock changes, customer and seller lookups, sales reporting and order history. My portfolio follows this same case study through relational design, query tuning, transaction control, document modelling and cloud distribution.

GITHUB Link: https://github.com/odigieosimen/advanced-database-portfolio

Portfolio 1 Relational design and data quality

I built a MySQL schema with nine related tables for customers, orders, items, payments, reviews, products, sellers, category translations and geolocation. Primary and foreign keys protect relationships, while constraints reject invalid values. I separated order items and payments because combining them in one flat table can multiply rows and overstate totals. The design reaches third normal form for most transactional tables. I documented specific exceptions where the supplied data prevented stricter normalization, including customer postcodes absent from the geolocation table. The loaded database contains about 99,441 orders and 1,000,163 geolocation rows. The ER model and row-count screenshots support the schema and load checks.

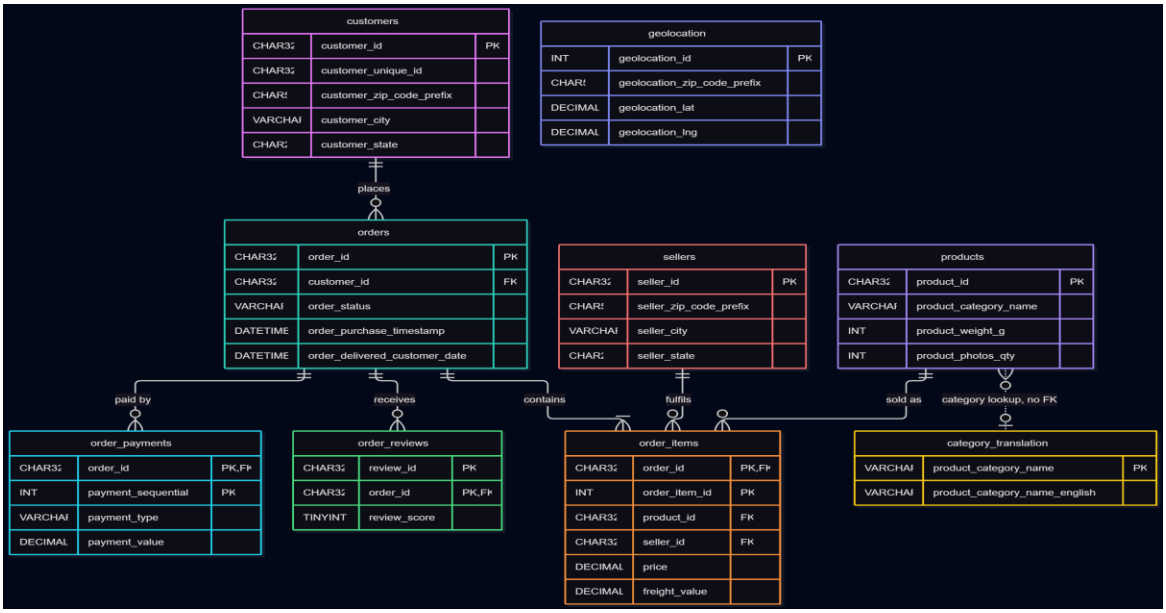


Figure 1. Relational entities and key relationships in the marketplace model.*Figure 1. Relational entities and key relationships in the marketplace model.*

GITHUB Link: https://github.com/odigieosimen/advanced-database-portfolio/tree/main/P1-database-design

Portfolio 2 Query processing and optimisation

I tested eight reporting queries with MySQL EXPLAIN ANALYZE and added six indexes chosen for the filters, joins and grouping used by the workload. The clearest example was the one-postcode geolocation query: a covering index reduced the rows examined from one million to 103, with median execution time falling from 1.88 seconds to about 0.17 milliseconds on this machine. Results varied by query. A control query that had no relevant new index did not improve on its first run, although later warm runs were faster. This showed me that caching can look like an indexing gain. I therefore interpreted timings alongside execution plans and marked the machine-specific results as indicative rather than universal.

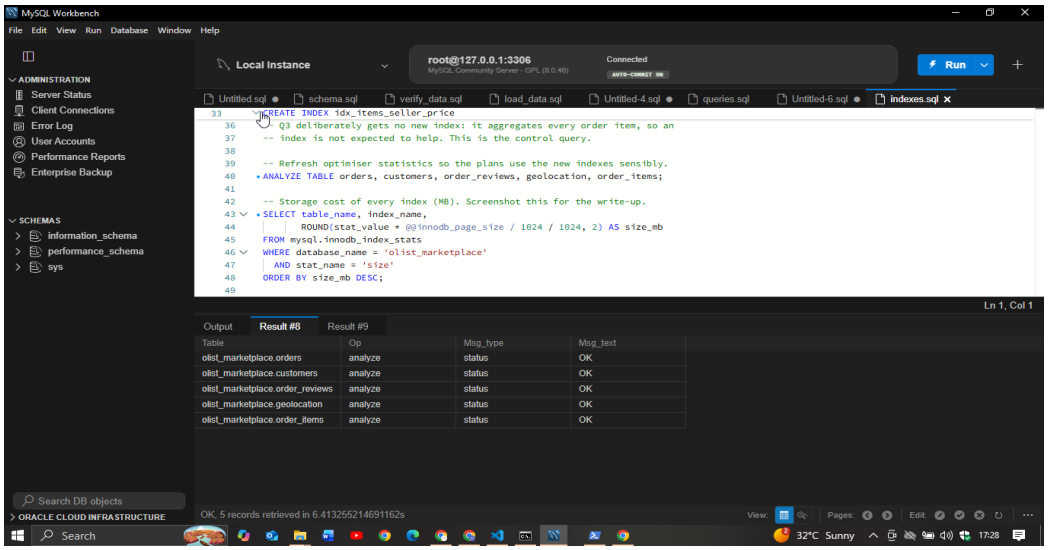


Figure 2. MySQL output confirming creation of the workload indexes.*Figure 2. MySQL output confirming creation of the workload indexes.*

GITHUB Link: https://github.com/odigieosimen/advanced-database-portfolio/tree/main/P2-query-optimisation

Portfolio 3 Transactions and concurrency

I added a small stock table to model checkout because stock quantities are not present in the source dataset. With InnoDB, I demonstrated that a failed checkout can roll back its order, item and stock changes together. Two-session tests showed a dirty read under READ UNCOMMITTED, a non-repeatable read under READ COMMITTED and snapshot behaviour under REPEATABLE READ. I also reproduced a lost update: two sessions each sold from a stock value of ten, but the final value became nine instead of eight. SELECT FOR UPDATE and an atomic conditional UPDATE prevented that error. A separate deadlock test showed InnoDB detecting a lock cycle and rolling back one transaction. These exercises made clear that a correct stock decision must be checked and applied as one protected operation.

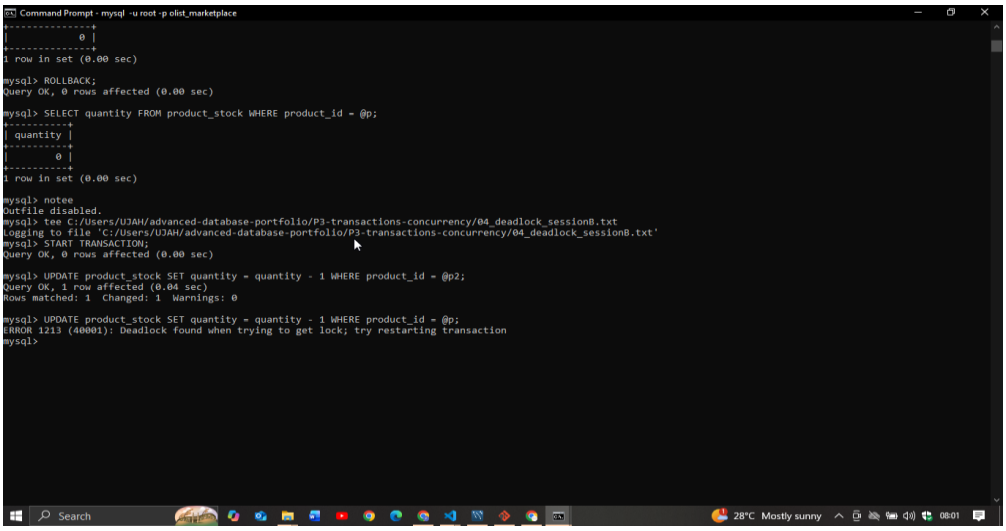


Figure 3. Session evidence of InnoDB identifying and resolving a deadlock.*Figure 3. Session evidence of InnoDB identifying and resolving a deadlock.*

GITHUB Link: https://github.com/odigieosimen/advanced-database-portfolio/tree/main/P3-transactions-concurrency

Portfolio 4 NoSQL document model

I exported the relational data to JSON and loaded it into MongoDB Atlas. Each order document embeds its bounded items, payments, reviews and a customer snapshot, so a common order-history read does not need several joins. Products and sellers remain separate because they are shared by many orders. I queried arrays and used aggregation pipelines for revenue, reviews and seller results. A compound index supported cancelled-order date searches, and schema validation rejected an invalid status. The orders collection contains a 20,000-order sample, while product and seller collections contain all records. This model makes complete order reads convenient, but cross-document integrity and reports spanning many entities remain easier to manage relationally.

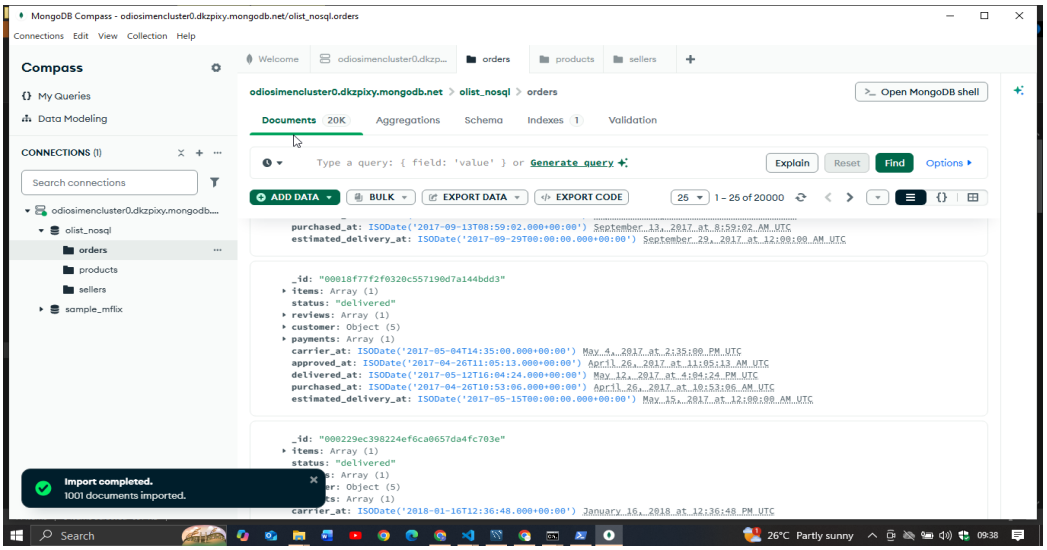


Figure 4. MongoDB Compass evidence of the imported order documents.*Figure 4. MongoDB Compass evidence of the imported order documents.*

GITHUB Link: https://github.com/odigieosimen/advanced-database-portfolio/tree/main/P4-nosql

Portfolio 5 Cloud distribution and design

I deployed the MongoDB document model to an AWS Paris Atlas free cluster and observed a three-member replica set with one primary and two secondaries. In the write-concern runs, writes from Lagos took roughly 185 milliseconds on average. The measured difference between w:1 and majority was too small to separate from run-to-run variation, so these tests do not establish that both guarantees have identical costs in other conditions. At the measured document size, the full order data is estimated at about 80 MB. Sharding is therefore unnecessary for this dataset. If growth later justified it, I recommend a hashed customer unique ID key because it should distribute the observed customer workload while keeping one customer's order history on one shard. Sharding and failover were not run on the free tier; these remain reasoned design proposals.

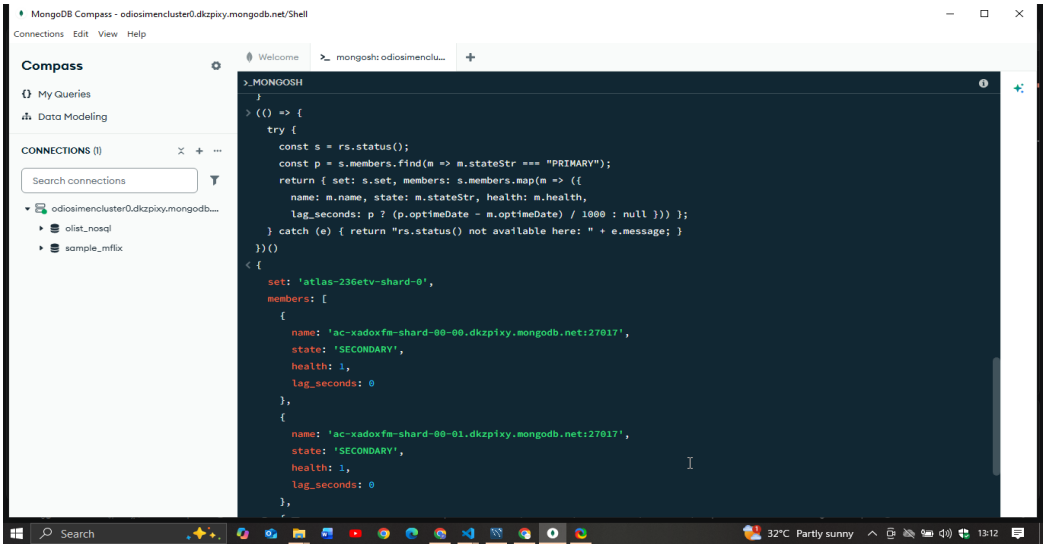


Figure 5. Atlas replica-set health evidence from the deployed cluster.*Figure 5. Atlas replica-set health evidence from the deployed cluster.*

GITHUB Link: https://github.com/odigieosimen/advanced-database-portfolio/tree/main/P5-distributed-cloud

Technical reflection

The most useful lesson for me was that database choices depend on how the data is read and changed. Normalization reduced repeated facts in MySQL, while embedding selected order data made MongoDB reads simpler. Neither design is automatically best for every task. The transaction exercises were especially useful because the lost update looked harmless in a single session but produced the wrong stock count under concurrency. I also learned to be careful with performance claims: a plan can show fewer rows examined, while elapsed time is still affected by cache state, network distance and the machine. My next improvement would be to run isolated repeated benchmarks and to test the proposed cloud design on a tier that permits failover and sharding.

Repository, references and AI use

The scripts, analyses, raw plans and supporting screenshots are organized in the project repository: https://github.com/odigieosimen/advanced-database-portfolio . The case study uses the Brazilian E-Commerce Public Dataset by Olist, distributed through Kaggle under CC BY-NC-SA 4.0. Technical references include the MySQL 8.0 Reference Manual, MongoDB Manual and MongoDB Atlas documentation. Tools used in the implementation include MySQL Community Server, MySQL Workbench, MongoDB Compass and MongoDB Atlas.

AI use declaration: OpenAI Codex assisted in organizing evidence and drafting this summary from the assignment brief and portfolio files. Copilot was used as a coding assistant inside VS Code. Claude was used to verify and validate SQL queries for MongoDB Atlas via Compass and MySQL Workbench.